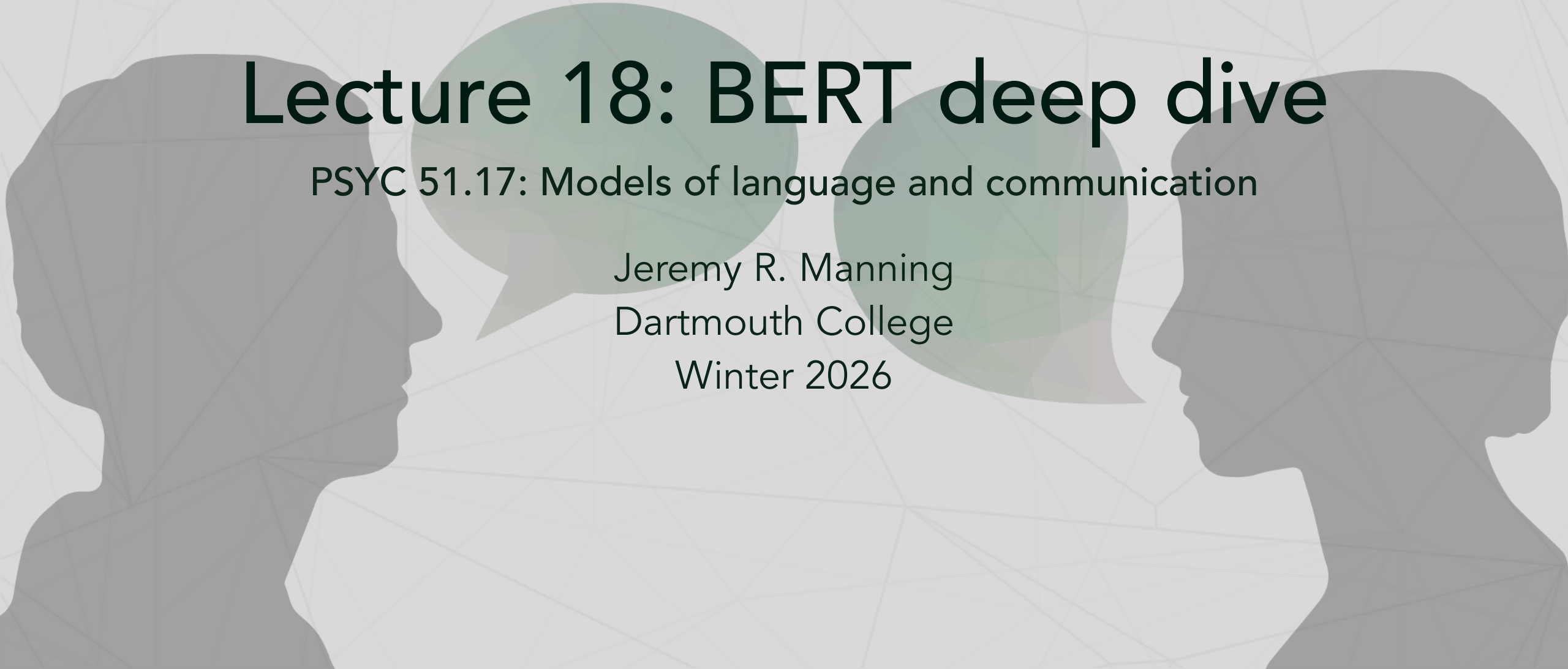
Lecture 18: BERT deep dive

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026



Learning objectives

By the end of this lecture, you will be able to...

1. Explain why bidirectional context matters (e.g., predicting `[MASK]` in "The robot `[MASK]` pancakes")
2. Describe the masked language modeling (MLM) training objective and the 80/10/10 strategy
3. Compare BERT-Base and BERT-Large architectures
4. Understand the pre-train then fine-tune paradigm
5. Demonstrate how contextual embeddings handle polysemy

Where we left off

Recap from lectures 15–16

In lecture 15, we built up the full transformer architecture step by step: tokenization → embeddings → Q/K/V → attention → feed-forward → prediction. The key idea was $\text{Transformer}(X, \theta) \to Y$, predicting the *next* token from previous tokens.

That was a **decoder-only** (GPT-style) transformer with **causal masking** — each token could only attend to tokens *before* it.

Today's running example

"The robot `[MASK]` pancakes"

What word goes in the blank? You probably guessed something like "flipped" or "cooked" — but only because you could read *both sides*. That's exactly what BERT does!

What is BERT?

Bidirectional Encoder Representations from Transformers

BERT is an **encoder-only transformer** that reads text in both directions simultaneously. Unlike autoregressive models (GPT), BERT sees the full context — both left and right — when processing each token.

The key innovation is **Masked Language Modeling (MLM)**: randomly mask tokens in the input and train the model to predict them from surrounding context.

Why bidirectional matters for our example

- **GPT** sees: "The robot ___" → could be *anything*: "destroyed", "ate", "purchased", "analyzed"...
- **BERT** sees: "The robot ___ pancakes" → "flipped" or "cooked" — the right context narrows it dramatically!

Causal vs. bidirectional attention

The key difference is one line of math

Recall from lecture 15 that attention scores are:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{H}}\right)$$

GPT adds a causal mask M before softmax, where $M_{ij} = -\infty$ if $j > i$:

$$A_{\text{GPT}} = \text{softmax}\left(\frac{QK^T}{\sqrt{H}} + M_{\text{causal}}\right)$$

BERT simply removes the mask — all positions attend to all positions:

$$A_{\text{BERT}} = \text{softmax}\left(\frac{QK^T}{\sqrt{H}}\right)$$

Trade-off

Removing the causal mask means BERT **cannot generate text** autoregressively — it sees the future! But it excels at *understanding* tasks.

Causal vs. bidirectional attention

Why bidirectionality matters

Walking through our example

"The robot `[MASK]` pancakes"

Direction	Context available	Likely predictions
Left only (GPT)	"The robot ___"	destroyed, ate, purchased, built, analyzed, ...
Right only	"___ pancakes"	flipped, stacked, burned, decorated, ...
Both (BERT)	"The robot ___ pancakes"	**flipped**, cooked, made — much more constrained!

Encoder vs. decoder

- **Encoder-only** (BERT) → *understanding* tasks: classification, NER, question answering
- **Decoder-only** (GPT) → *generation* tasks: text completion, dialogue, code generation
- **Encoder-decoder** (T5, BART) → *sequence-to-sequence*: translation, summarization

Why BERT was revolutionary

BERT changed everything about NLP (2018)

Before BERT:

- Feature-based approaches (Word2Vec/GloVe as fixed features)
- Task-specific architectures for each problem
- Limited transfer learning — models trained from scratch
- Unidirectional or shallow bidirectional models (ELMo)

BERT's contributions:

1. **Deep bidirectionality** — true bidirectional context at every layer, not just concatenating left-to-right and right-to-left
2. **Pre-train + fine-tune paradigm** — one pre-trained model adapted to many tasks with minimal changes
3. **State-of-the-art results** — beat previous best on 11 NLP benchmarks, sometimes by 10+ points

BERT input representation

Three embeddings summed together

BERT's input is the element-wise sum of three embedding types, each producing a vector of size C:

$$E_{\text{input}} = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}}$$

1. **Token embeddings**: WordPiece vocabulary lookup (30K learned vectors)
2. **Segment embeddings**: Which sentence — A or B? (2 learned vectors)
3. **Position embeddings**: Learned position encoding for positions 0–511

Compare with lecture 15

GPT uses only token + position embeddings. BERT adds **segment embeddings** to handle sentence-pair tasks like question answering and natural language inference.

BERT input representation

CLS	A	0
the	A	1
rob	A	2
MSK	A	3
pan	A	4
SEP	A	5

BERT input representation for our example

Input representation for our running example

```
tokens      = ["[CLS]", "the", "robot", "[MASK]", "pancakes", "[SEP]"]
token_emb   = [E_CLS,   E_the, E_robot, E_MASK,   E_pancakes, E_SEP]
segment_emb = [E_A,     E_A,   E_A,     E_A,      E_A,        E_A  ]
position_emb= [E_0,     E_1,   E_2,     E_3,      E_4,        E_5  ]

# Final input = token_emb + segment_emb + position_emb (element-wise)
```

Dimensions

Each embedding vector has size $C = 768$ (for BERT-Base). With $T = 6$ tokens, the combined input matrix is $T \times C = 6 \times 768$.

Segment embeddings

Handling sentence pairs

For tasks involving two sentences (e.g., question answering, natural language inference), BERT needs to know which tokens belong to which sentence. **Segment embeddings** solve this:

- All tokens in **Sentence A** get embedding vector E_A
- All tokens in **Sentence B** get embedding vector E_B
- Only 2 learned vectors — extremely parameter-efficient!

Sentence pair example

Sentence A: "The robot flipped pancakes" Sentence B: "They were delicious"

Token	[CLS]	the	robot	flippe d	panca kes	[SEP]	they	were	delicio us	[SEP]
Segme nt	A	A	A	A	A	A	B	B	B	B

Segment embeddings

WordPiece tokenization

How BERT handles unknown words

```python
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Common words stay intact
tokenizer.tokenize("The cat sat on the mat")
# → ['the', 'cat', 'sat', 'on', 'the', 'mat']

# Rare words get split into subwords
tokenizer.tokenize("unbelievably")
# → ['un', '##believable', '##ly']  # "##" means continuation

tokenizer.tokenize("ChatGPT is transformative")
# → ['chat', '##g', '##pt', 'is', 'transform', '##ative']
```

Masked language modeling

BERT's primary pre-training objective

Training procedure:

1. Take a sentence from the training corpus
2. Randomly select 15% of tokens for prediction
3. Of those selected tokens:
   - **80%** are replaced with `[MASK]`
   - **10%** are replaced with a random word
   - **10%** are kept unchanged
4. Train the model to predict the original tokens

Why the 80/10/10 split?

If we always used `[MASK]`, the model would learn to only pay attention when it sees `[MASK]` — but `[MASK]` never appears in real text during fine-tuning. The random replacement and unchanged tokens force the model to maintain good representations for *all* tokens, not just masked ones.

MLM step by step

Walkthrough with our running example

```
# Step 1: Start with the original sentence
tokens = ["[CLS]", "the", "robot", "flipped", "pancakes", "[SEP]"]
# Select "flipped" (idx 3) for prediction (1 of 6 tokens ≈ 15%)

# Step 2: Apply the 80/10/10 strategy
# "flipped" → 80% chance → replaced with [MASK]

# Step 3: Create training example
input:  "[CLS] the robot [MASK] pancakes [SEP]"
labels: [-1,   -1,  -1,    "flipped", -1,        -1]
# -1 means no loss computed at this position

# Step 4: Model predicts
```

Masked language modeling

MLM loss function

Formally...

The masked language modeling loss is the negative log-likelihood of predicting the original tokens at masked positions:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P(x_i \mid \mathbf{x}_{\setminus \mathcal{M}}; \theta)$$

where:

- $\mathcal{M}$ = set of masked positions
- $\mathbf{x}_{\setminus \mathcal{M}}$ = input with masked tokens
- θ = model parameters
- Only masked positions contribute to the loss (the -1 labels are ignored)

Prediction at each masked position

$$P(x_i = w \mid \mathbf{x}_{\setminus \mathcal{M}}) = \text{softmax}(\mathbf{h}_i \cdot W_{\text{vocab}} + b)_w$$

where $\mathbf{h}_i$ is BERT's output hidden state at position i.

Next sentence prediction

BERT's second pre-training objective

Task: Given two sentences A and B, predict whether B actually follows A in the original text.

- **50% of the time:** B is the real next sentence (label: `IsNext`)
- **50% of the time:** B is a random sentence from the corpus (label: `NotNext`)

The `[CLS]` token representation is used for this binary classification.

NSP is controversial

Later work (RoBERTa, 2019) showed that removing NSP actually *improves* performance. The task may be too easy — distinguishing topic is simpler than understanding sentence relationships. ALBERT replaced NSP with the harder **Sentence Order Prediction** (SOP) task.

BERT making predictions

Predicting from [MASK], not from the last token

Recall from lecture 15: GPT uses the **last token's** output vector to predict the next word.

BERT is different — it predicts from **each masked position's** output vector:

$$P(\text{"flipped"} \mid \text{context}) = \text{softmax}(\mathbf{h}_{[\text{MASK}]} \cdot W_{\text{vocab}} + b)$$

The output hidden state $\mathbf{h}_{[\text{MASK}]}$ at the `[MASK]` position encodes information from *all* surrounding tokens (both left and right).

For classification tasks

For sentence-level tasks (sentiment, NLI), BERT uses the `[CLS]` token's output instead — this token is trained to aggregate information across the whole sequence.

BERT making predictions

BERT architecture variants

Two model sizes

Component	BERT-Base	BERT-Large
Transformer layers	12	24
Hidden size	768	1024
Attention heads	12	16
Feed-forward size	3072	4096
Total parameters	110M	340M
Max sequence length	512 tokens	512 tokens
Vocabulary	30,000 (WordPiece)	30,000 (WordPiece)

Special tokens

- **[CLS]**: Classification token (first position, used for sequence-level tasks)
- **[SEP]**: Separator token (between sentence pairs)
- **[MASK]**: Mask token (for MLM pre-training)
- **[PAD]**: Padding token (for batching variable-length sequences)

BERT pre-training

Massive-scale pre-training on unlabeled text

Pre-training data:

- **BooksCorpus**: 800M words (11,038 unpublished books)
- **English Wikipedia**: 2,500M words (text only, no tables/lists/headers)
- **Total**: 3.3 billion words of diverse, high-quality text

Training details:

- Batch size: 256 sequences (128,000 tokens per batch)
- Training steps: 1 million
- Optimizer: Adam (lr = 1e-4, warmup over first 10K steps)
- Hardware: 4–16 Cloud TPUs
- Training time: ~4 days for BERT-Base

Key insight

Pre-training learns *general language understanding* that transfers to many

The pre-train then fine-tune paradigm

Two-stage training

Stage 1 — Pre-training (done once, expensive):

- Data: 3.3B words of unlabeled text
- Objective: MLM + NSP
- Cost: Days on TPU clusters
- Result: General-purpose language representations

Stage 2 — Fine-tuning (per task, cheap):

- Data: 1K–100K labeled examples for your specific task
- Objective: Task-specific loss (e.g., cross-entropy for classification)
- Cost: Hours on a single GPU
- Result: Task-specialized model with strong performance

Why this works

Pre-training captures syntax, semantics, and world knowledge from massive text.

The pre-train then fine-tune paradigm

Fine-tuning for different tasks

Minimal architecture changes needed

Task type	Input format	Output	Example
Single sentence classification	[CLS] sentence [SEP]	[CLS] → classifier	Sentiment analysis
Sentence pair classification	[CLS] sent-A [SEP] sent-B [SEP]	[CLS] → classifier	Natural language inference
Question answering	[CLS] question [SEP] passage [SEP]	Token-level start/end positions	SQuAD
Token classification	[CLS] sentence [SEP]	Each token → classifier	Named entity recognition

The same pre-trained BERT is used for all tasks — only a simple output layer is added on top.

Fine-tuning BERT in Python

Using HuggingFace Transformers for sentiment classification

```
from transformers import BertForSequenceClassification, Trainer, TrainingArguments

# Load pre-trained BERT with a classification head
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2  # Binary classification (positive/negative)
)

# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,         # Much lower than pre-training!
```

continued...

Fine-tuning BERT in Python

Using HuggingFace Transformers for sentiment classification

```
    warmup_steps=500,
)

# Train
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)
trainer.train()
```

...continued

Contextual embeddings in action

The word 'bank' gets different embeddings depending on context

```
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

sent1 = "I deposited money at the bank"    # Financial
institution
sent2 = "We sat by the river bank"          # Riverbank

def get_embedding(sentence, target_word):
    inputs = tokenizer(sentence, return_tensors='pt')
    outputs = model(**inputs)
```

continued...

Contextual embeddings in action

The word 'bank' gets different embeddings depending on context

```
        tokens = tokenizer.tokenize(sentence)
        idx = tokens.index(target_word) + 1    # +1 for [CLS]
        return outputs.last_hidden_state[0, idx, :]

emb1 = get_embedding(sent1, "bank")  # Financial context
emb2 = get_embedding(sent2, "bank")  # River context

similarity = torch.cosine_similarity(emb1, emb2, dim=0)
print(f"Similarity: {similarity:.3f}")  # ~0.3–0.5 (low! different
meanings)
```

...continued

Static vs. contextual embeddings

BERT captures meaning differences that static embeddings miss

Word pair	Word2Vec similarity	BERT similarity
bank (financial) vs bank (river)	1.00 (same vector!)	~0.42
bank (financial) vs money	0.65	~0.78
bank (river) vs shore	0.52	~0.81

Why this matters

Word2Vec and GloVe assign a *single* vector to each word, regardless of context. BERT generates a *different* vector for each occurrence of a word, shaped by its surrounding context. This is why BERT excels at tasks requiring disambiguation — it actually "sees" the difference between financial banks and riverbanks.

BERT's benchmark results

State-of-the-art on 11 NLP tasks when released (2018)

Task	Metric	Previous SOTA	BERT-Large
SQuAD 2.0 (question answering)	F1	66.3	83.1
MNLI (natural language inference)	Accuracy	80.6	86.7
SST-2 (sentiment analysis)	Accuracy	93.2	94.9
CoNLL-2003 (named entity recognition)	F1	92.6	92.8

Key observations:

- Largest gains on tasks requiring deep understanding (QA, NLI)
- Improvements even on well-studied, heavily-optimized benchmarks
- BERT-Large consistently outperforms BERT-Base

Impact

BERT made pre-trained transformers the default starting point in NLP. Nearly all subsequent models — RoBERTa, ALBERT, DistilBERT, GPT-2 — build on ideas BERT popularized.

What does BERT learn?

Probing BERT's internal representations

Research has shown that BERT's layers form a linguistic processing pipeline:

1. **Lower layers (1–4)**: Surface features — part-of-speech tags, word boundaries, morphology
2. **Middle layers (5–8)**: Syntactic structure — parse trees, dependency relations, subject-verb agreement
3. **Upper layers (9–12)**: Semantics and pragmatics — word sense disambiguation, coreference, entity types
4. **Final layers**: Task-specific representations that emerge during fine-tuning

Analogy to vision

This is similar to how convolutional neural networks learn: early layers detect edges, middle layers detect shapes, and later layers detect objects. BERT does the same thing with language — from characters to meaning.

Layer analysis with probing

Extracting representations from different layers

```
from transformers import BertModel

model = BertModel.from_pretrained('bert-base-uncased', output_hidden_states=True)
outputs = model(**inputs)
hidden_states = outputs.hidden_states  # 13 tensors: embedding + 12 layers

# Results from probing studies (Tenney et al., 2019):
layer_specialization = {
    "Layers 0-2":  ["POS tagging", "Word boundaries"],  # Surface
    "Layers 3-6":  ["Parse trees", "Dependencies"],  # Syntax
    "Layers 7-9":  ["Semantic roles", "Coreference"],
```

Discussion

Questions to consider

1. **MLM vs autoregressive modeling**: Why is MLM better for *understanding* tasks? Could BERT generate text like GPT? What are the trade-offs?
2. **The 80/10/10 masking strategy**: Why not use 100% `[MASK]` replacement? What problem does the random word replacement solve? Could we improve this strategy?
3. **Pre-training data choices**: Why use books and Wikipedia? Would social media text work as well? How does data quality affect what BERT learns?
4. **Fine-tuning efficiency**: Why does fine-tuning work so well with so little data? When might it fail? How much labeled data do we actually need?

References

Further reading

Devlin et al. (2019, *NAACL*) "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" — The original BERT paper.

Tenney et al. (2019, *ACL*) "BERT Rediscovers the Classical NLP Pipeline" — Layer-by-layer analysis of what BERT learns.

Clark et al. (2019, *BlackboxNLP*) "What Does BERT Look At? An Analysis of BERT's Attention" — Attention pattern analysis.

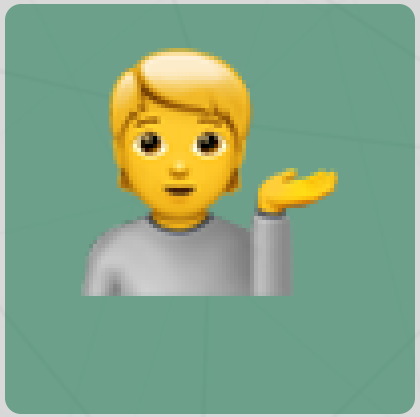
HuggingFace Course, Chapter 1 — Practical introduction to transformer models.

Jay Alammar: The Illustrated BERT — Visual walkthrough of BERT's architecture.

Questions?

Email

Discord



Office hours

Up next...

BERT variants: how RoBERTa, ALBERT, DistilBERT, and ELECTRA improved on the original